



UNIVERSIDAD PRIVADA DOMINGO SAVIO

Sistema Web y Móvil con Tecnología NFC para la Gestión del Pasaje Estudiantil en Cochabamba

Integrantes:

Joaquin Alessandro Felipez Rojas

Nicolas Sebastian Reguerin Meneses

David Willy Cruz Huanca

Docente:

Alejandra Gómez Mendoza

Fecha de entrega: 07 de abril de 2026

Índice

1. Introducción	1
1.1. Antecedentes	1
1.2. Planteamiento del problema	2
1.2.1. Objetivo general	2
1.2.2. Objetivos específicos	2
2. Marco conceptual	4
2.1. Arquitecturas de aplicaciones web	4
2.1.1. Aplicación web	4
2.1.2. Modelo cliente-servidor	4
2.1.3. Protocolo HTTP	4
2.2. Tecnologías del lado del servidor	5
2.3. Entorno de desarrollo local	5
2.3.1. Servidor local	5
2.3.2. PHP	5
2.3.3. MySQL	6
2.4. Intercambio de datos en aplicaciones web	6
2.4.1. Formato JSON	6
2.4.2. Web Service	6
2.5. Tecnologías de interacción cliente-servidor	7

2.6. Framework	7
2.6.1. Laravel	7
2.6.2. NextJs	7
2.6.3. React	8
2.6.4. Tailwind CSS	8
2.7. Arquitectura MVC	8
2.8. API	8
2.9. Docker	9
2.10. JavaScript	9
2.11. TypeScript	9
3. Marco práctico	10
3.1. Instalación y configuración del entorno	10
3.1.1. Instalación del entorno local	10
3.1.2. Instalación de PHP	10
3.1.3. Instalación de Node.js	10
3.1.4. Instalación de Docker Engine	11
3.1.5. Instalación de los frameworks	12
3.1.5.1. Instalación del framework backend (Laravel)	12
3.1.5.2. Instalación del framework frontend (Next.js)	13
3.1.6. Configuración de la base de datos	14
3.2. Arquitectura del sistema	15

3.2.1. Arquitectura cliente-servidor	15
3.2.2. Patrón MVC en el backend	15
3.3. Estructura del proyecto	16
3.3.1. Organización de carpetas	16
3.3.2. Directorios principales	16
3.4. Diseño de la base de datos	17
3.4.1. Diagrama conceptual	17
3.4.2. Diagrama lógico	18
3.4.3. Diagrama físico	19
3.5. Desarrollo de la aplicación	19
3.5.1. Definición de rutas e integración de la API	19
3.5.1.1. Enrutamiento en el Backend (Laravel)	19
3.5.1.2. Consumo en el Frontend (Next.js)	20
3.5.2. Creación de controladores	21
3.5.3. Creación de modelos	22
3.5.4. Desarrollo de vistas y componentes de interfaz	23
3.5.4.1. Estrategia de renderizado (Server y Client Components)	23
3.5.4.2. Sistema de diseño y librería UI	23
3.5.4.3. Estructuración de Vistas Complejas	23
3.5.5. Implementación de funcionalidades	26
3.6. Interacción con la base de datos	27

3.6.1. Configuración de conexión	27
3.6.2. Consultas realizadas	28
3.6.3. Manipulación de datos	28
3.6.4. Creación de registros	28
3.6.4.1. Eliminación simulada (Soft Delete)	29
3.7. Validación y manejo de errores	29
3.7.1. Validación de formularios	29
3.7.2. Manejo de excepciones	30
3.7.3. Control de errores	30
3.8. Pruebas del sistema	31
3.8.1. Pruebas de funcionamiento	31
3.9. Resultados	34
3.9.1. Funcionamiento final	34
3.9.2. Capturas del sistema	35
4. Conclusiones y recomendaciones	39

Índice de figuras

1.1. Árbol de problemas del sistema de cobro del transporte estudiantil . .	2
3.1. Instalación de PHP y dependencias en Debian.	10
3.2. Instalación de Node.js en el entorno local.	11
3.3. Instalación de Docker Engine en el entorno local (Debian).	12
3.4. Inicialización del proyecto backend en Laravel 11.	13
3.5. Inicialización del proyecto web con Next.js.	14
3.6. Ejecución de migraciones de MySQL en Laravel.	15
3.7. Diagrama conceptual del sistema.	17
3.8. Diagrama lógico del sistema.	18
3.9. Diagrama físico de la base de datos.	19
3.10.Arquitectura de comunicación y flujo de datos entre el Frontend y el Backend.	21
3.11.Jerarquía de componentes y composición de una vista administrativa.	25
3.12.Interfaz de usuario de la vista de gestión de usuarios.	26
3.13.Configuración del servicio MySQL en Docker Compose	27
3.14.Filtrado y consulta mediante Eloquent	28
3.15.Inserción de datos en la tabla Usuario	29
3.16.Lógica de eliminación lógica	29
3.17.Verificación de respuesta exitosa (200 OK) y estructura JSON en el <i>endpoint</i> de usuarios.	31

3.18. Interfaz del <i>dashboard</i> de usuarios mostrando el funcionamiento de filtros dinámicos y paginación.	32
3.19. Visualización de la interfaz en entorno móvil con el menú lateral colapsado automáticamente.	33
3.20. Diagrama de flujo de validación de datos entre cliente y servidor. . . .	34
3.21. Interfaz de autenticación del sistema, con validación de credenciales en tiempo real.	35
3.22. Módulo de transacciones (<i>Data Table</i>) mostrando el listado de operaciones, con soporte para búsqueda dinámica y paginación.	36
3.23. Panel interactivo para la gestión de roles, demostrando el uso de modales laterales (<i>Sheets</i>) para la inserción de datos sin abandonar el contexto de la tabla.	37
3.24. Vista de gestión de dispositivos móviles y tarjetas NFC vinculadas a las cuentas de los usuarios.	38

Índice de tablas

3.1. Variables de configuración de la base de datos	27
---	----

1. Introducción

El transporte público en el municipio de Cochabamba representa un pilar fundamental para la movilidad urbana diaria, siendo el sector estudiantil uno de sus usuarios más frecuentes. Para garantizar el acceso a la educación, normativas municipales establecen una tarifa preferencial y diferenciada para los estudiantes de nivel primario y secundario. Sin embargo, el método tradicional de cobro en efectivo dentro de minibuses y trufis genera constantes fricciones, retrasos y un cumplimiento irregular de dicha tarifa.

El presente proyecto propone la modernización y digitalización de este proceso mediante el diseño y desarrollo de un ecosistema tecnológico cliente-servidor. La solución integra un sistema web (Backend) encargado de la gestión de usuarios y saldos virtuales, junto con una aplicación móvil Android que funciona como terminal de cobro (lector) y un sistema de tarjetas físicas provistas de tecnología NFC para los estudiantes. A través de este modelo, el sistema permite que el estudiante realice el pago de su pasaje de manera rápida, segura y totalmente offline al acercar su tarjeta al dispositivo del conductor, quien valida la transacción y recibe los fondos correspondientes en su cuenta virtual. De esta manera, se busca eliminar el uso de dinero en efectivo, garantizar el respeto a la tarifa estudiantil y proveer un registro auditable de los viajes realizados.

1.1 Antecedentes

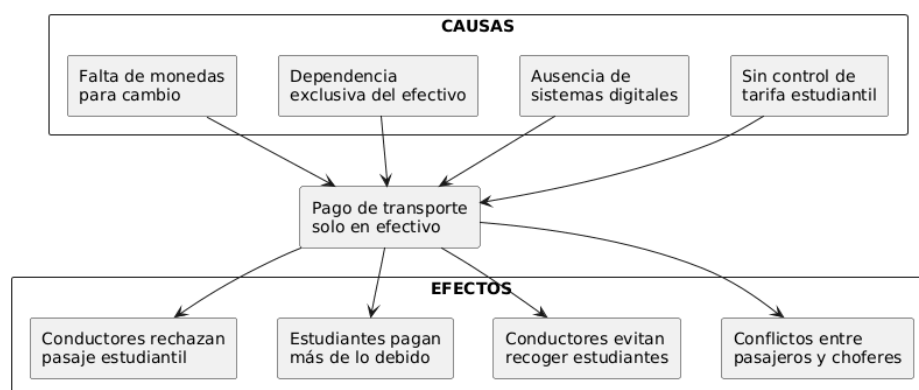
Históricamente, en la ciudad de Cochabamba y a nivel nacional, el pago del transporte público descentralizado (sindicatos de trufis y minibuses) se ha realizado exclusivamente mediante dinero en efectivo. Si bien existen normativas municipales que amparan el derecho de los estudiantes a pagar una tarifa reducida, la aplicación práctica de esta medida presenta graves deficiencias. Es un escenario común que los conductores del transporte público se nieguen a cobrar el pasaje diferenciado argumentando la falta de cambio (monedas de baja denominación), o que directamente eviten recoger a los estudiantes en horarios pico para no percibir ingresos menores.

A nivel tecnológico, en Bolivia existen referentes exitosos de modernización del transporte masivo, como el sistema "Mi Teleférico." los buses "PumaKatari."^{en} La Paz,

los cuales utilizan tarjetas físicas RFID/NFC para el cobro automatizado con un alto grado de eficiencia y aceptación ciudadana. El uso de tarjetas físicas ha demostrado ser el método más accesible y rápido para el abordaje de pasajeros. Al adaptar este modelo exitoso al transporte descentralizado de Cochabamba, y complementarlo con el uso de los teléfonos inteligentes de los conductores como terminales de validación, se democratiza el acceso a los pagos digitales sin requerir instalaciones de hardware costosas dentro de cada vehículo.

1.2 Planteamiento del problema

Figura 1.1: Árbol de problemas del sistema de cobro del transporte estudiantil



Fuente: Elaboración propia

1.2.1 Objetivo general

Desarrollar un sistema web y una aplicación móvil Android, integrando tecnología NFC, para la gestión y cobro eficiente del pasaje estudiantil en la ciudad de Cochabamba.

1.2.2 Objetivos específicos

- Elaborar el modelo relacional de la base de datos para gestionar de forma íntegra y segura los perfiles de los usuarios (estudiantes y choferes), los saldos virtuales y el registro histórico de las transacciones.

- Diseñar el *backend* del sistema en PHP bajo el patrón de arquitectura MVC, implementando un módulo de administración (CRUD) de usuarios y una API para el procesamiento de peticiones de cobro.
- Implementar un mecanismo de seguridad basado en tokenización (como JWT) para generar credenciales temporales, permitiendo que el pago desde el dispositivo del estudiante se realice de manera *online*.
- Programar el módulo de la aplicación móvil Android dirigido al estudiante, utilizando la tecnología de Emulación de Tarjeta Host (HCE) para transmitir de forma segura el token identificador vía NFC.
- Construir la interfaz (móvil o web) dirigida al chofer, encargada de activar el modo lector NFC, capturar el token del estudiante y comunicarse con el servidor para validar y ejecutar el descuento de la tarifa preferencial.
- Ejecutar pruebas de funcionamiento e integración entre las aplicaciones cliente y el servidor en un entorno local, verificando la velocidad de respuesta, la integridad de los saldos y el correcto manejo de errores.

2. Marco conceptual

2.1 Arquitecturas de aplicaciones web

Según e (r), la arquitectura de una aplicación web define la estructura general del sistema, estableciendo la manera en que los distintos componentes interactúan entre sí para proporcionar servicios al usuario final a través de la red.

En el presente proyecto, la arquitectura web es fundamental debido a que el sistema propuesto se basa en un modelo cliente-servidor. La aplicación web funcionará como el backend encargado de gestionar usuarios, transacciones y saldos virtuales del sistema de pago del transporte estudiantil.

2.1.1 Aplicación web

De acuerdo con r (u), una aplicación web es un software que se ejecuta en un servidor y que los usuarios pueden acceder mediante un navegador web utilizando protocolos de comunicación como HTTP.

En el proyecto propuesto, la aplicación web constituye el núcleo del sistema, ya que será responsable de gestionar la información relacionada con los estudiantes, conductores y las transacciones de pago realizadas mediante la tecnología NFC.

2.1.2 Modelo cliente-servidor

Según n (a), el modelo cliente-servidor es una arquitectura de red en la cual múltiples clientes solicitan servicios o recursos a un servidor central que procesa las solicitudes y responde con la información correspondiente.

En el sistema planteado, el dispositivo móvil del conductor y el sistema del estudiante actuarán como clientes que interactúan con el servidor central donde se almacenará la base de datos y se procesarán las operaciones de validación de pago del pasaje estudiantil.

2.1.3 Protocolo HTTP

Según r (u), el protocolo HTTP (HyperText Transfer Protocol) es el protocolo de comunicación utilizado para la transferencia de información entre clientes y servidores dentro de la World Wide Web.

En el desarrollo del sistema propuesto, HTTP permitirá que la aplicación móvil y la interfaz web se comuniquen con el servidor mediante solicitudes y respuestas que permitirán consultar información, validar usuarios y registrar las transacciones realizadas.

2.2 Tecnologías del lado del servidor

De acuerdo con e (r), las tecnologías del lado del servidor son aquellas que permiten procesar la lógica de negocio de una aplicación web, gestionar bases de datos y generar respuestas dinámicas que son enviadas al cliente.

En el presente proyecto, las tecnologías del lado del servidor permitirán gestionar la información de los usuarios del sistema, controlar el saldo de las tarjetas estudiantiles y registrar cada transacción realizada durante el uso del transporte público.

2.3 Entorno de desarrollo local

Según e (r), un entorno de desarrollo local permite a los desarrolladores probar y ejecutar aplicaciones en su propio equipo antes de desplegarlas en un servidor de producción.

Para el desarrollo del sistema propuesto se utilizará un entorno de desarrollo local que permitirá simular el funcionamiento completo del sistema web antes de su implementación en un entorno real.

2.3.1 Servidor local

Un servidor local es un servidor que se ejecuta en el propio equipo del desarrollador con el objetivo de probar aplicaciones web sin necesidad de conexión a internet (n,

a).

En el desarrollo del proyecto, el servidor local permitirá ejecutar el sistema web, realizar pruebas de funcionamiento y validar la comunicación entre la aplicación móvil y el backend del sistema.

2.3.2 PHP

Según 2 (,), PHP es un lenguaje de programación del lado del servidor ampliamente utilizado para el desarrollo de aplicaciones web dinámicas.

En este proyecto, PHP será utilizado para desarrollar el backend del sistema, implementar la lógica de negocio y gestionar la comunicación con la base de datos.

2.3.3 MySQL

De acuerdo con 2 (,), MySQL es un sistema de gestión de bases de datos relacional que permite almacenar, organizar y consultar grandes cantidades de información de manera eficiente.

En el sistema propuesto, MySQL será utilizado para almacenar la información de los usuarios, tarjetas estudiantiles y registros de transacciones realizadas.

2.4 Intercambio de datos en aplicaciones web

El intercambio de datos es un aspecto fundamental en las aplicaciones web modernas, ya que permite la comunicación entre diferentes sistemas mediante formatos estandarizados (r, u).

En el proyecto desarrollado, el intercambio de datos permitirá que la aplicación móvil y el servidor web compartan información relacionada con usuarios, saldos y registros de transacciones.

2.4.1 Formato JSON

Según 2 (.), JSON es un formato ligero de intercambio de datos basado en texto que es fácil de leer y escribir para los humanos y sencillo de procesar por las máquinas.

En el sistema propuesto, JSON será utilizado como formato principal para enviar y recibir datos entre la aplicación móvil y el servidor web.

2.4.2 Web Service

Según I (r), un Web Service es un conjunto de funciones accesibles a través de la red que permiten la comunicación entre diferentes aplicaciones mediante protocolos estándar.

En el proyecto propuesto, se implementará una API que funcionará como un servicio web encargado de procesar las solicitudes de validación y cobro del pasaje estudiantil.

2.5 Tecnologías de interacción cliente-servidor

Las tecnologías de interacción cliente-servidor permiten mejorar la experiencia del usuario al facilitar la comunicación dinámica entre el navegador y el servidor sin necesidad de recargar completamente la página (r, a).

Estas tecnologías serán utilizadas en el sistema web para permitir consultas dinámicas de información y mejorar la eficiencia de la interacción con el sistema.

2.6 Framework

De acuerdo con e (r), un framework es una estructura de software que proporciona herramientas y componentes reutilizables que facilitan el desarrollo de aplicaciones.

En el desarrollo del sistema propuesto, el uso de un framework permitirá organizar el código del proyecto y mejorar su mantenibilidad.

2.6.1 Laravel

Según 2 (,), Laravel es un framework de desarrollo web para PHP que proporciona herramientas modernas para la construcción de aplicaciones robustas y escalables.

En el proyecto propuesto, Laravel permitirá implementar de forma estructurada la lógica del sistema, facilitando la gestión de rutas, controladores y modelos.

2.6.2 NextJs

De acuerdo con 2 (,), Next.js es un framework de desarrollo basado en React que permite la creación de aplicaciones web optimizadas mediante renderizado del lado del servidor y generación de sitios estáticos.

En el sistema propuesto, Next.js será utilizado para estructurar el frontend de la aplicación, mejorando el rendimiento y la organización del código, así como facilitando la integración con la API del backend.

2.6.3 React

Según 2 (,), React es una biblioteca de JavaScript para construir interfaces de usuario basadas en componentes reutilizables y un flujo de datos unidireccional.

En el proyecto propuesto, React será utilizado para desarrollar la interfaz del sistema web, permitiendo crear componentes reutilizables que faciliten la visualización dinámica de la información de usuarios, saldos y transacciones.

2.6.4 Tailwind CSS

Según 2 (,), Tailwind CSS es un framework de CSS basado en utilidades que permite construir interfaces personalizadas mediante clases predefinidas.

En el proyecto propuesto, Tailwind CSS será utilizado para diseñar la interfaz del sistema web, permitiendo un desarrollo más rápido y consistente de los estilos visuales de la aplicación.

2.7 Arquitectura MVC

De acuerdo con m (a), el patrón Modelo-Vista-Controlador (MVC) separa la lógica de negocio, la interfaz de usuario y el control de la aplicación en diferentes componentes.

En el sistema desarrollado, la arquitectura MVC permitirá organizar el código de manera clara, separando la gestión de datos, la lógica del sistema y la interfaz de usuario.

2.8 API

Según e (i), una API (Interfaz de Programación de Aplicaciones) es un conjunto de definiciones y protocolos que permiten la comunicación entre diferentes sistemas de software.

En el proyecto desarrollado, la API permitirá la interacción entre la aplicación móvil, el sistema web y el backend, gestionando las solicitudes relacionadas con autenticación, validación de pagos y consulta de datos.

2.9 Docker

De acuerdo con 2 (.), Docker es una plataforma que permite desarrollar, enviar y ejecutar aplicaciones dentro de contenedores, asegurando consistencia en diferentes entornos de desarrollo y producción.

En el sistema propuesto, Docker será utilizado para gestionar el entorno de desarrollo, facilitando la configuración del servidor, la base de datos y otros servicios necesarios para el funcionamiento del sistema.

2.10 JavaScript

Según 2 (.), JavaScript es un lenguaje de programación interpretado, ampliamente utilizado para el desarrollo de aplicaciones web interactivas tanto en el cliente como en el servidor.

En el proyecto propuesto, JavaScript será utilizado para implementar la lógica del lado del cliente, permitiendo la interacción dinámica con el usuario y la comunicación con el backend mediante solicitudes HTTP.

2.11 TypeScript

De acuerdo con [2], TypeScript es un superconjunto de JavaScript que añade tipado estático opcional, mejorando la detección de errores y la mantenibilidad del código.

En el sistema desarrollado, TypeScript será utilizado para mejorar la calidad del código en el frontend, facilitando el desarrollo de componentes más robustos y escalables.

3. Marco práctico

3.1 Instalación y configuración del entorno

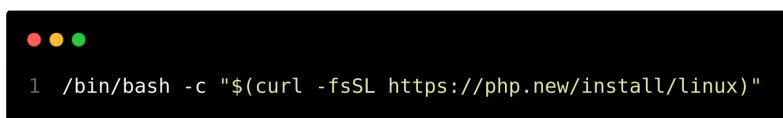
3.1.1 Instalación del entorno local

Para el desarrollo del proyecto, se estableció un entorno de trabajo nativo sobre el sistema operativo Debian Linux. La preparación de este entorno requirió la instalación individual de las herramientas base para el funcionamiento del backend, el frontend y la base de datos.

3.1.2 Instalación de PHP

Dado que el backend del sistema está desarrollado en Laravel, fue necesaria la instalación de PHP en el entorno local, junto con sus extensiones requeridas y el gestor de dependencias Composer. Esto permite ejecutar el servidor de desarrollo y gestionar los paquetes de la API RESTful de manera nativa sobre Debian. En la Figura ?? se muestran los comandos ejecutados para este propósito.

Figura 3.1: Instalación de PHP y dependencias en Debian.

A terminal window with a black background and three colored window control buttons (red, yellow, green) in the top-left corner. The terminal displays a single command: `1 /bin/bash -c "$(curl -fsSL https://php.new/install/linux)"`.

```
1 /bin/bash -c "$(curl -fsSL https://php.new/install/linux)"
```

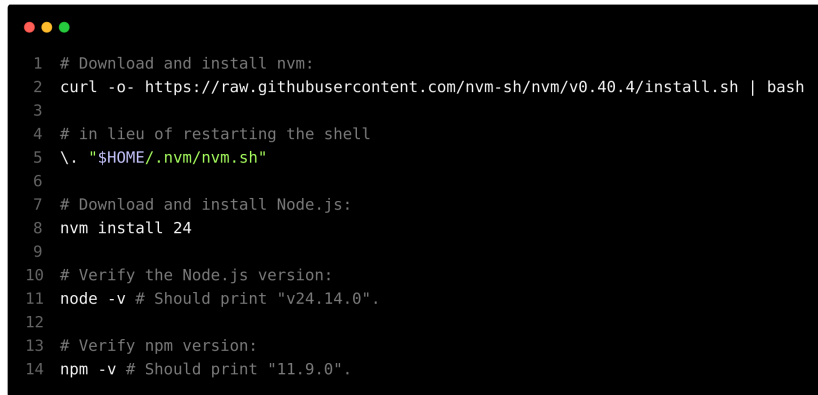
Fuente: Elaboración propia.

3.1.3 Instalación de Node.js

Para la construcción y ejecución del frontend web desarrollado con Next.js, se procedió a la instalación del entorno de ejecución Node.js y su gestor de paquetes (npm). La instalación se realizó para garantizar la compatibilidad requerida por las herramientas modernas de React y el *App Router*. La Figura ?? detalla el proceso

de instalación en la terminal.

Figura 3.2: Instalación de Node.js en el entorno local.

A terminal window with a black background and white text. The text shows a series of commands for installing Node.js using nvm. The commands are numbered 1 through 14. The output of the commands is shown in green text. The commands are: 1. # Download and install nvm: 2. curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.4/install.sh | bash 3. 4. # in lieu of restarting the shell 5. \. "\$HOME/.nvm/nvm.sh" 6. 7. # Download and install Node.js: 8. nvm install 24 9. 10. # Verify the Node.js version: 11. node -v # Should print "v24.14.0". 12. 13. # Verify npm version: 14. npm -v # Should print "11.9.0".

```
1 # Download and install nvm:
2 curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.4/install.sh | bash
3
4 # in lieu of restarting the shell
5 \. "$HOME/.nvm/nvm.sh"
6
7 # Download and install Node.js:
8 nvm install 24
9
10 # Verify the Node.js version:
11 node -v # Should print "v24.14.0".
12
13 # Verify npm version:
14 npm -v # Should print "11.9.0".
```

Fuente: Elaboración propia.

3.1.4 Instalación de Docker Engine

Para la capa de persistencia de datos, se optó por aislar el motor de base de datos MySQL utilizando contenedores. Esto evita conflictos de versiones con otras aplicaciones del sistema anfitrión y mantiene el entorno limpio. Siguiendo la documentación oficial para Debian, se instaló Docker Engine. La Figura ?? detalla los comandos utilizados mediante el gestor de paquetes apt para la configuración inicial del motor.

Figura 3.3: Instalación de Docker Engine en el entorno local (Debian).

```
1 # Add Docker's official GPG key:
2 sudo apt update
3 sudo apt install ca-certificates curl
4 sudo install -m 0755 -d /etc/apt/keyrings
5 sudo curl -fsSL https://download.docker.com/linux/debian/gpg -o /etc/apt/keyrings/docker.asc
6 sudo chmod a+r /etc/apt/keyrings/docker.asc
7
8 # Add the repository to Apt sources:
9 sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
10 Types: deb
11 URIs: https://download.docker.com/linux/debian
12 Suites: $(. /etc/os-release && echo "$VERSION_CODENAME")
13 Components: stable
14 Signed-By: /etc/apt/keyrings/docker.asc
15 EOF
16
17 sudo apt update
```

Fuente: Elaboración propia.

3.1.5 Instalación de los frameworks

El proyecto adopta un enfoque de desarrollo desacoplado, separando la lógica de negocio de la interfaz de usuario. Para lograr esto, se implementaron dos frameworks distintos que operan de manera independiente pero se comunican a través de una API RESTful. Ambos entornos se inicializaron de manera nativa sobre el sistema operativo Debian, aprovechando las herramientas de desarrollo locales configuradas previamente (PHP y Node.js). A continuación, se detalla el proceso de inicialización de cada uno.

3.1.5.1 Instalación del framework backend (Laravel)

Para la construcción del backend, encargado de gestionar la validación de las transacciones NFC y la lógica de negocio, se instaló la versión 11 del framework Laravel. Esta instalación se llevó a cabo utilizando Composer, el gestor de dependencias de PHP. Mediante la terminal, se ejecutó el comando de creación de proyecto, el cual descargó el núcleo del framework y generó la estructura base de directorios. En la Figura ?? se ilustra este proceso.

Figura 3.4: Inicialización del proyecto backend en Laravel 11.A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. It displays a list of 13 numbered commands for initializing a Laravel 11 backend project.

```
1 # Instalar laravel
2 composer global require laravel/installer
3
4 # Crear el nuevo proyecto
5
6 laravel new example-app
7
8
9 # Inicializar el proyecto
10
11 cd backend
12 npm install && npm run build
13 composer run dev
```

Fuente: Elaboración propia.

3.1.5.2 Instalación del framework frontend (Next.js)

Por otro lado, la interfaz web (cliente administrativo) se inicializó utilizando Next.js. Aprovechando el entorno de Node.js, se ejecutó la utilidad `create-next-app` para generar la estructura del proyecto web. Durante la instalación, se configuró el proyecto para utilizar el *App Router*, asegurando un entorno de desarrollo moderno para el consumo de la API. La Figura ?? muestra el comando ejecutado para la creación de la aplicación frontend.

Figura 3.5: Inicialización del proyecto web con Next.js.A terminal window with a black background and three colored window control buttons (red, yellow, green) in the top-left corner. It contains three lines of white text representing terminal commands.

```
1 npx create next-app@latest frontend --yes
2 cd frontend
3 npm run dev
```

Fuente: Elaboración propia.

3.1.6 Configuración de la base de datos

La persistencia de datos se resolvió utilizando MySQL. A diferencia de un entorno completamente contenedorizado, Docker se empleó de manera exclusiva para levantar la instancia de MySQL, lo que garantiza que el motor de base de datos se ejecute en un entorno limpio y reproducible sin afectar los recursos del sistema anfitrión. Para ello, se definió la configuración del servicio en el archivo `docker-compose.yml` (ver Anexo A), ubicado en la raíz del proyecto, y se inicializó el contenedor ejecutando el comando `docker compose up -d` en la terminal.

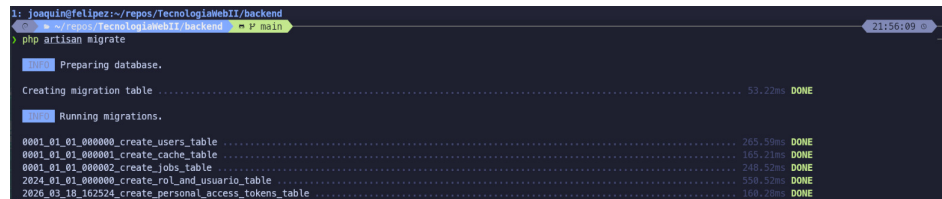
Una vez que el servicio de base de datos estuvo en ejecución, se procedió a establecer la conexión con el backend nativo de Laravel. Esto se logró modificando el archivo de configuración `.env` dentro del directorio `/backend`, donde se definieron las credenciales y parámetros de red mediante las siguientes variables de entorno:

- `DB_CONNECTION=mysql`
- `DB_HOST=127.0.0.1`
- `DB_PORT=3306`
- `DB_DATABASE=laravel_db`
- `DB_USERNAME=laravel_user`
- `DB_PASSWORD=secret_password`

Finalmente, con la conexión a la base de datos exitosamente establecida, se procedió

a estructurar el esquema de datos. Para ello, se ejecutó el comando `php artisan migrate` en el entorno de desarrollo de Laravel. Esta acción procesó los archivos de migración del framework, creando las tablas necesarias para gestionar los usuarios, tarjetas NFC y el historial de pasajes estudiantiles. La salida de este proceso se puede observar en la Figura ??.

Figura 3.6: Ejecución de migraciones de MySQL en Laravel.



```

i: joaquin@felipez:~/repos/TecnologiaWebII/backend
└─> php artisan migrate
    Preparing database.
    Creating migration table ..... 53.22ms DONE
    Running migrations.
    0001_01_01_000000_create_users_table ..... 265.59ms DONE
    0001_01_01_000001_create_cache_table ..... 165.21ms DONE
    0001_01_01_000002_create_jobs_table ..... 248.52ms DONE
    2024_01_01_000000_createRol_and_usuario_table ..... 558.52ms DONE
    2026_03_10_162524_create_personal_access_tokens_table ..... 168.29ms DONE
  
```

Nota. Salida de la terminal al ejecutar `php artisan migrate`.

Fuente: Elaboración propia.

3.2 Arquitectura del sistema

3.2.1 Arquitectura cliente-servidor

El sistema implementa una arquitectura cliente-servidor estricta, operando en repositorios y puertos separados. El servidor (backend en Laravel) actúa exclusivamente como un proveedor de servicios mediante una API, gestionando la lógica de negocio, la validación de las transacciones NFC para el pasaje estudiantil y la interacción con MySQL. El cliente web (frontend en Next.js) consume estos servicios de forma asíncrona y se encarga de la renderización de la interfaz administrativa e interactiva.

3.2.2 Patrón MVC en el backend

Dentro del ecosistema del backend, Laravel implementa el patrón arquitectónico Modelo-Vista-Controlador (MVC). Dada la naturaleza de API del proyecto, la capa de "Vista" devuelve exclusivamente respuestas estructuradas en formato JSON.

- **Modelos** (`app/Models`): Representan las entidades de datos (como Usuarios, Tarjetas NFC, Transacciones) y gestionan la relación con MySQL a través del ORM Eloquent.

- **Controladores** (`app/Http/Controllers`): Procesan las peticiones HTTP entrantes desde la aplicación Next.js y los dispositivos móviles, interactúan con los modelos y devuelven el estado de las operaciones.

3.3 Estructura del proyecto

3.3.1 Organización de carpetas

El proyecto está organizado dividiendo claramente el sistema en dos entornos separados: el *backend* (API) y el *frontend* (interfaz de usuario). En la raíz del proyecto se encuentra el archivo `docker-compose.yml` (destinado al despliegue de los servicios y la base de datos), acompañado de dos directorios principales.

3.3.2 Directorios principales

A continuación, se describe la función de los directorios más relevantes de la aplicación:

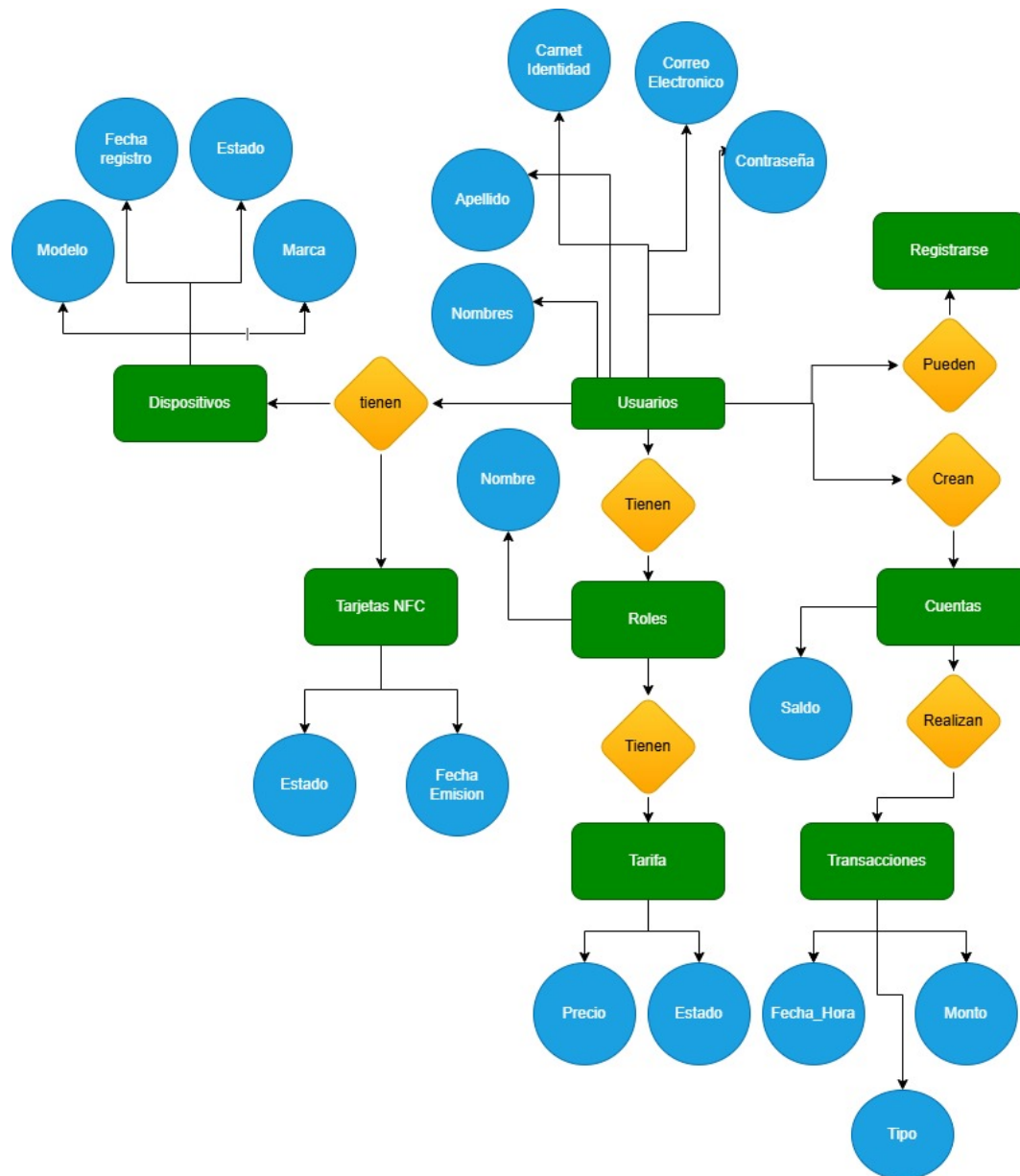
- `/backend`: Contiene el núcleo de la API desarrollada en Laravel. Destacan las carpetas `app/` (lógica de negocio y modelos), `config/` (configuraciones globales), `database/` (migraciones de la base de datos) y `routes/` (donde `api.php` define los *endpoints* del sistema).
- `/frontend`: Alberga la aplicación web desarrollada con Next.js, la cual está estructurada bajo una arquitectura modular orientada a dominios:
 - `app/`: Define el sistema de enrutamiento mediante el *App Router*. Separa las rutas públicas de autenticación dentro del grupo (`auth`) y las vistas administrativas protegidas dentro de `dashboard/` (cuentas, dispositivos, roles, transacciones y usuario).
 - `components/`: Aloja los componentes visuales compartidos de la interfaz, incluyendo la navegación y el subdirectorio `ui/` para componentes base reutilizables.
 - `modules/`: Constituye el núcleo de la lógica del frontend. Agrupa el código por dominios específicos (`auth`, `cuentas`, `roles`, etc.), encapsulando internamente sus propios componentes visuales, *hooks*, esquemas de validación y servicios de conexión (*queries* a la API).
 - `lib/` e `hooks/`: Contienen utilidades transversales. `lib/` almacena configuracio-

nes esenciales como los clientes de conexión (`api-client.ts`), y `hooks/` guarda ganchos globales de la interfaz.

3.4 Diseño de la base de datos

3.4.1 Diagrama conceptual

Figura 3.7: Diagrama conceptual del sistema.

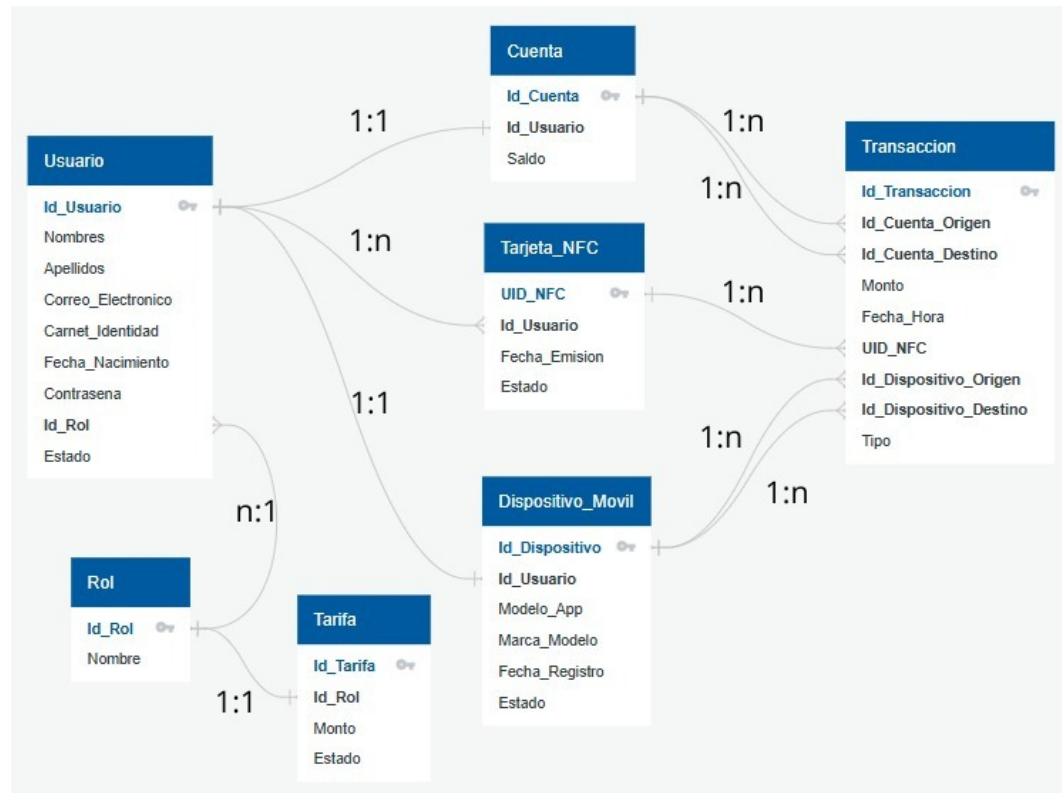


Nota. Representa las entidades principales y sus relaciones a nivel abstracto, sin detalles de implementación.

Fuente: Elaboración propia.

3.4.2 Diagrama lógico

Figura 3.8: Diagrama lógico del sistema.

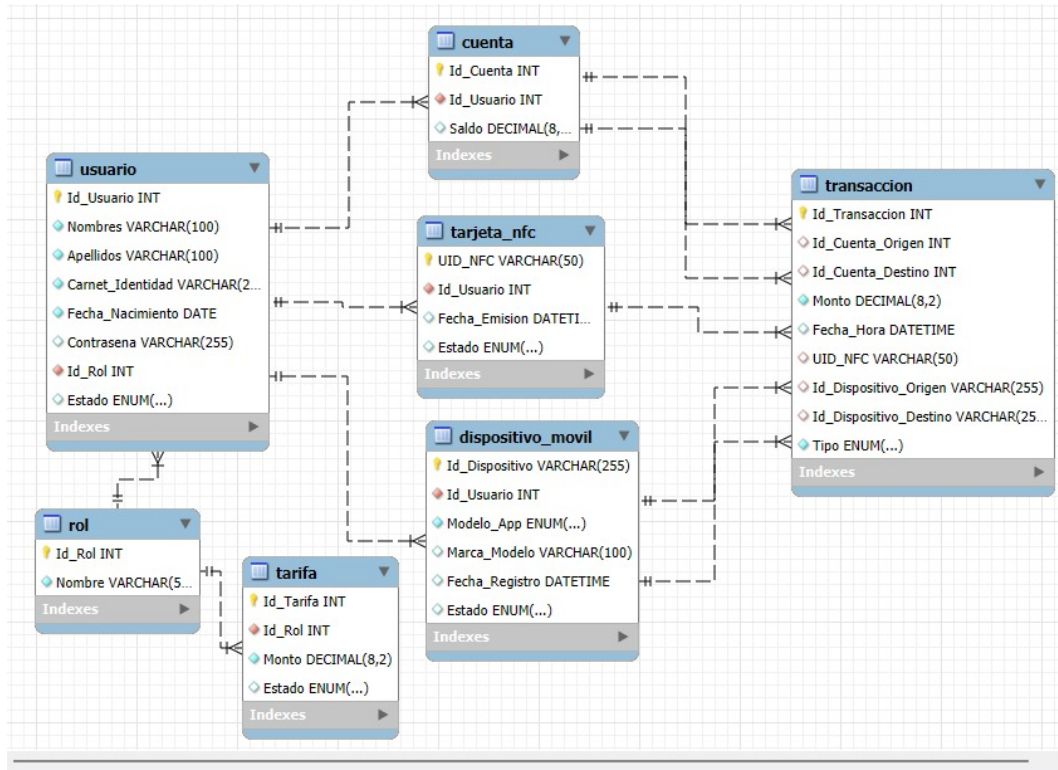


Nota. Describe la estructura de datos, atributos y relaciones, considerando reglas de negocio y normalización.

Fuente: Elaboración propia.

3.4.3 Diagrama físico

Figura 3.9: Diagrama físico de la base de datos.



Nota. Implementación en el gestor de base de datos, incluyendo tipos de datos, claves primarias, foráneas y restricciones.

Fuente: Elaboración propia.

3.5 Desarrollo de la aplicación

3.5.1 Definición de rutas e integración de la API

El sistema implementa una arquitectura basada en servicios (API RESTful) que comunica el *backend* (Laravel) con el *frontend* (Next.js). La definición de rutas abarca desde la exposición de *endpoints* en el servidor hasta su respectivo consumo y validación en el cliente.

3.5.1.1 Enrutamiento en el Backend (Laravel)

La configuración de rutas se centraliza en el archivo `routes/api.php`. El diseño de la API sigue las siguientes convenciones principales:

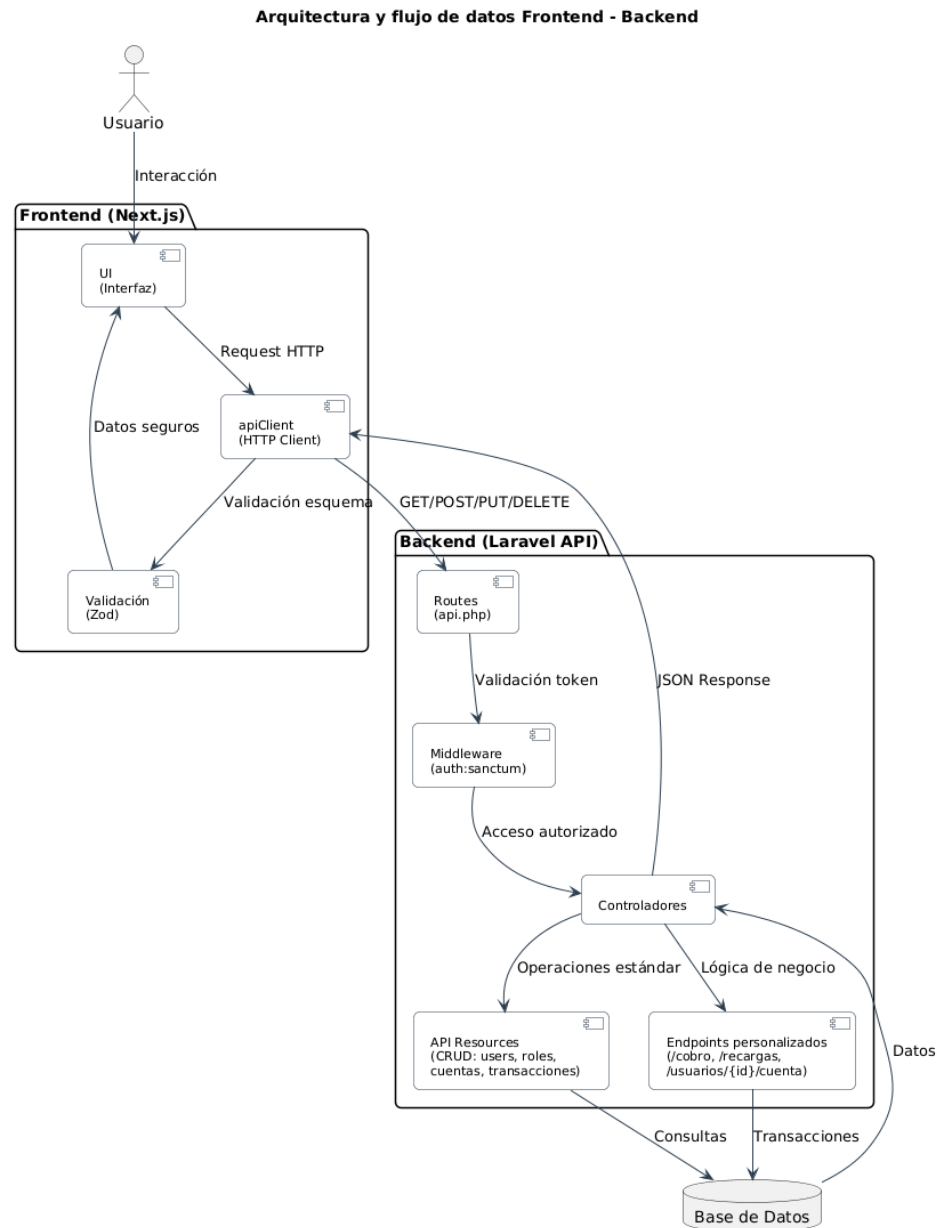
- **Autenticación y Seguridad:** Se exponen rutas públicas para el registro (`/register`) y acceso (`/login`). Por otro lado, las rutas que devuelven la sesión del usuario (`/user`) o ejecutan cierres de sesión están protegidas bajo el *middleware* `auth:sanctum`, garantizando que solo clientes con un *token* válido accedan.
- **Controladores de Recursos (API Resources):** Se utiliza el método `Route::apiResource` para generar automáticamente las rutas CRUD (Crear, Leer, Actualizar, Eliminar) de entidades clave, tales como `users`, `roles`, `tarifas`, `cuentas` y `transacciones`.
- **Rutas Específicas de Lógica de Negocio:** Para procesos que no encajan en el modelo CRUD estándar, se definen *endpoints* personalizados. Esto incluye la gestión transaccional (ej. `/cobro`, `/recargas`) y la obtención de relaciones complejas (ej. `/usuarios/{id}/cuenta`).

3.5.1.2 Consumo en el Frontend (Next.js)

En el lado del cliente, las peticiones HTTP se abstraen dentro de los módulos correspondientes. Tomando como ejemplo el servicio de usuarios (`usuario.api.ts`), el flujo de datos se maneja de la siguiente forma:

- **Cliente HTTP Centralizado:** Se emplea una instancia personalizada llamada `apiClient` que se encarga de estandarizar las peticiones (inyectando *headers* y manejando sesiones) hacia los *endpoints* de Laravel.
- **Manejo Dinámico de Parámetros:** Para peticiones GET complejas como listados (`fetchUsuarios`), se utiliza la interfaz `URLSearchParams`. Esto permite inyectar dinámicamente parámetros de paginación, filtros (*estado*, *rolld*), búsquedas (*search*) y ordenamiento directo en la URL de la API.
- **Validación de Tipos Estrictos:** Para garantizar la integridad de la información que ingresa al estado de la aplicación, las respuestas en formato JSON se validan en tiempo de ejecución utilizando la librería `Zod` (ej. `usuarioResponseSchema.parse(data)`). Si la respuesta del servidor no coincide con el esquema esperado, se intercepta el error para evitar fallos silenciosos en la interfaz.

Figura 3.10: Arquitectura de comunicación y flujo de datos entre el Frontend y el Backend.



Fuente: Elaboración propia.

3.5.2 Creación de controladores

UserController: Para registrar, editar, suspender (Inactivo) a estudiantes, choferes y cajeros. Al crear un usuario, el controlador también debería crear automáticamente

su Cuenta con Saldo 0.

TarifaController: Para modificar los precios. (Si mañana la alcaldía sube el pasaje de 0.50 a 0.60, lo cambias aquí y todo el sistema se actualiza).

TarjetaNfcController: Para asignar una tarjeta física (leer el UID_NFC) y enlazarla al Id_Usuario de un estudiante. También para cambiar su estado a 'Perdida' si el estudiante la reporta.

RecargaController (o CajaController): La interfaz web donde el cajero recibe dinero en efectivo de un estudiante, busca su cuenta y le aumenta el Saldo (Generando una transaccion de tipo 'Recarga Saldo' con un Id_Cuenta_Destino nulo).

ReporteController: Para ver métricas globales. ¿Cuánto se recaudó hoy? ¿Cuántos pasajes se pagaron?

RolController: Para crear, borrar, actualizar y leer los diferentes roles.

AuthController: Para que el chofer introduzca su correo/carnet y contraseña para abrir su turno en el celular. (Los estudiantes usarán esto si tienen una app para ver su saldo).

CobroController: En la App Android del chofer lee una tarjeta y manda el UID_NFC por POST a este controlador.

Lógica paso a paso:

- Busca la Tarjeta_NFC → verifica que esté 'Activa' → saca al Usuario dueño.
- Saca su Cuenta → saca el Rol → busca la Tarifa para ese rol.
- Descuenta el Monto del Saldo origen → lo suma al Saldo destino.
- Registra la Transaccion → devuelve un JSON:

```
{"status": ".ok", "msj": "Cobro exitoso", "saldo_restante": 12.50}
```

CuentaController: Para tener un endpoint (ej. GET /api/cuenta/mi-saldo) donde un usuario pueda ver ya accedeetalles de su cuenta

3.5.3 Creación de modelos

Rol: Relaciones: hasMany(Usuario::class), hasMany(Tarifa::class)

Tarifa: Relaciones: belongsTo(Rol::class) (Ej: Te dirá que el pasaje de 0.50 Bs le pertenece al rol 'Estudiante').

Usuario: Relaciones: belongsTo(Rol::class), hasOne(Cuenta::class) (Billetera única por persona), hasMany(TarjetaNFC::class) (Por si pierde una tarjeta y le dan otra).

Cuenta: Relaciones: belongsTo(Usuario::class), hasMany(Transaccion::class)

(Como cuenta origen o destino).

TarjetaNFC: Relaciones: `belongsTo(Usuario::class)`, `hasMany(Transaccion::class)`.

Transaccion: Relaciones: `belongsTo(Cuenta::class, 'Id_Cuenta_Origen')` (Quien paga), `belongsTo(Cuenta::class, 'Id_Cuenta_Destino')` (El chofer/sindicato que recibe), `belongsTo(TarjetaNFC::class, 'UID_NFC')`.

3.5.4 Desarrollo de vistas y componentes de interfaz

El desarrollo de la capa visual del *frontend* se abordó mediante una arquitectura basada en componentes reutilizables, apoyándose en las convenciones del *App Router* de Next.js y un sistema de diseño estandarizado.

3.5.4.1 Estrategia de renderizado (Server y Client Components)

Para optimizar el rendimiento y la experiencia del usuario, se implementó una separación clara entre componentes de servidor y de cliente:

- **Server Components (por defecto):** Se utilizaron para las páginas estáticas y los *layouts* principales (como `app/dashboard/layout.tsx`). Esto permite que la estructura base de la aplicación se genere en el servidor, reduciendo el tamaño del paquete JavaScript enviado al navegador.
- **Client Components:** Aquellos elementos que requieren interactividad, manejo de estado local o el uso de *hooks* (como los formularios de registro, tablas de datos interactivos y modales) se marcaron explícitamente con la directiva `use client`. Un ejemplo de esto son los componentes dentro de `modules/usuario/components/`.

3.5.4.2 Sistema de diseño y librería UI

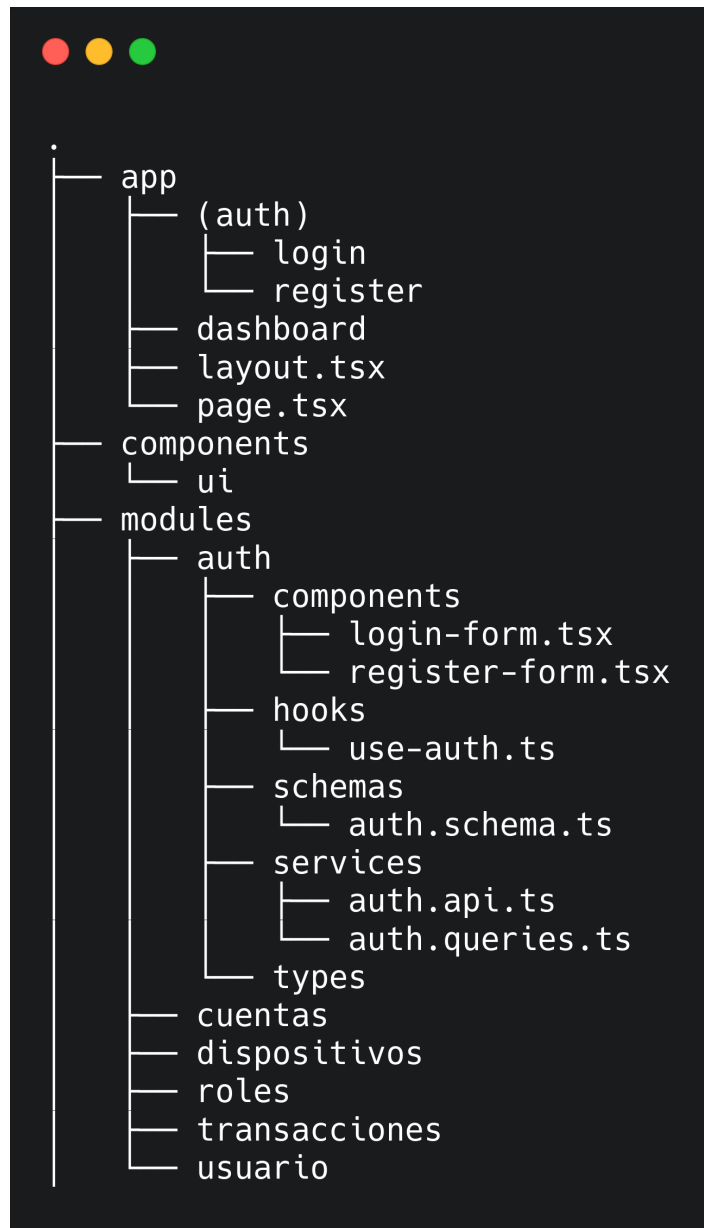
Para mantener la coherencia visual y acelerar el desarrollo, se integró **Tailwind CSS** junto con una arquitectura de componentes base accesibles. La carpeta `components/ui/` actúa como una biblioteca interna donde residen elementos atómicos (*botones, inputs, selectores, avatares, tablas y menús desplegables*). Estos componentes encapsulan sus propios estilos y comportamientos base, garantizando que vistas complejas, como los *dashboards* administrativos, compartan el mismo lenguaje visual.

3.5.4.3 Estructuración de Vistas Complejas

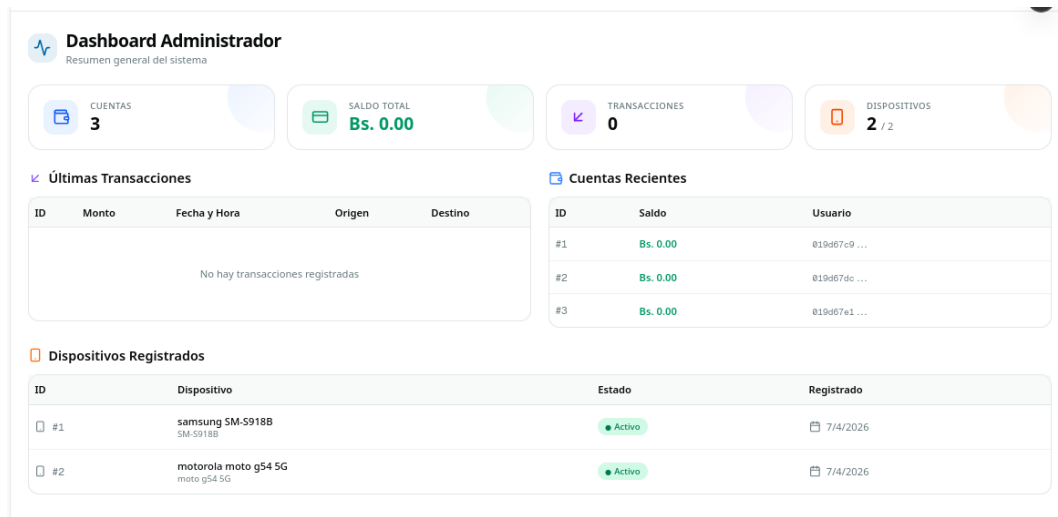
Las interfaces principales, como el panel de administración (*dashboard*), se componen de varias capas jerárquicas:

1. **Capa de Envoltorio (*Layouts*):** Proveen el esqueleto de navegación persistente, integrando componentes globales como `app-sidebar.tsx` y `nav-main.tsx`.
2. **Capa de Página (*Pages*):** Actúan como contenedores principales de cada ruta (ej. `cuentas/page.tsx`).
3. **Capa de Dominio (*Módulos*):** Las páginas delegan la presentación de la información a componentes especializados del dominio, como `cuentas-data-table.tsx`, los cuales gestionan la paginación, filtros y acciones directas sobre los registros.

Figura 3.11: Jerarquía de componentes y composición de una vista administrativa.



Fuente: Elaboración propia.

Figura 3.12: Interfaz de usuario de la vista de gestión de usuarios.

Fuente: Elaboración propia.

3.5.5 Implementación de funcionalidades

La funcionalidad del sistema reside en el `CobroController`, donde el flujo de intercambio de valor será mediante una transacción atómica de base de datos para garantizar la consistencia de los saldos. Se utilizan *API Resources* para estandarizar la salida JSON, el flujo algorítmico implementado la siguiente secuencia lógica:

1. **Validación de Identidad:** Se intercepta el `UID_NFC` enviado por el microcontrolador, verificando su existencia en el repositorio de `TarjetaNFC` y comprobando que su estado operativo sea 'Activa'.
2. **Resolución de Tarifa:** El sistema identifica el rol del usuario propietario de la tarjeta y consulta en la tabla `Tarifa` el monto vigente correspondiente a dicho perfil.
3. **Verificación de Solvencia:** Se realiza una validación lógica para confirmar que el saldo disponible en la `Cuenta` del estudiante es mayor o igual al monto de la tarifa obtenida.
4. **Ejecución de Transacción:** Mediante un bloque *database transaction*, se realiza simultáneamente el descuento (débito) de la cuenta del estudiante y el abono (crédito) a la cuenta del conductor/operador.
5. **Registro de Auditoría:** Se persiste un registro en la tabla `Transaccion`, capturando los IDs de los dispositivos móviles involucrados en la operación, asegurando la

trazabilidad forense del cobro.

3.6 Interacción con la base de datos

3.6.1 Configuración de conexión

La conexión con la base de datos MySQL en el entorno de desarrollo se gestiona a través del archivo de configuración de entorno `.env`. Laravel utiliza este archivo para centralizar las credenciales y parámetros de red. Además, para asegurar la portabilidad y limpieza del entorno, se utiliza Docker Engine para orquestar el contenedor de MySQL.

A continuación, en el Cuadro ?? se detallan las variables de entorno configuradas para el proyecto, seguidas por el fragmento del archivo `docker-compose.yml` (Figura) que define el servicio de persistencia y la herramienta de administración visual Adminer.

Cuadro 3.1: Variables de configuración de la base de datos

Variable	Valor configurado
DB_CONNECTION	mysql
DB_HOST	127.0.0.1
DB_PORT	3306
DB_DATABASE	laravel_db
DB_USERNAME	laravel_user
DB_PASSWORD	secret_password

Fuente: Elaboración propia.

Figura 3.13: Configuración del servicio MySQL en Docker Compose

```
services:
  db:
    image: mysql:8.0
    environment:
      MYSQL_DATABASE: laravel_db
      MYSQL_ROOT_PASSWORD: root_password
      MYSQL_USER: laravel_user
      MYSQL_PASSWORD: secret_password
    ports:
      - "3306:3306"
```

Fuente: Elaboración propia.

3.6.2 Consultas realizadas

Para la interacción con los datos, se utiliza el ORM Eloquent de Laravel, el cual permite realizar consultas a la base de datos mediante una sintaxis orientada a objetos en lugar de consultas SQL manuales. En el sistema de pasajes, las consultas más representativas incluyen filtros de búsqueda y paginación. Un ejemplo destacado es la recuperación de usuarios en el `UserController`, donde se implementa lógica para filtrar por nombres, apellidos o carnet de identidad, junto con un sistema de ordenamiento dinámico:

Figura 3.14: Filtrado y consulta mediante Eloquent

```
$query = User::query();

if ($request->has('search')) {
    $search = $request->search;
    $query->where(function($q) use ($search) {
        $q->where('Nombres', 'like', "
            %{$search}%")
        ->orWhere('Apellidos', 'like', "
            %{$search}%")
        ->orWhere('Carnet_Identidad',
            'like', "%{$search}%");
    });
}

$users = $query->paginate($request->get('
    perPage', 10));
```

Fuente: Elaboración propia.

3.6.3 Manipulación de datos

La manipulación de datos comprende las operaciones de creación, actualización y eliminación. En el proyecto, se prioriza la integridad de los datos mediante el uso de asignación masiva (*mass assignment*) y validación de reglas de negocio antes de persistir la información.

3.6.4 Creación de registros

Para registrar un nuevo usuario, se utiliza el método `create()`, el cual recibe un arreglo asociativo con los datos validados del estudiante o chofer. El sistema se encarga de aplicar el *hashing* a la contraseña para su almacenamiento seguro:

Figura 3.15: Inserción de datos en la tabla Usuario

```

$user = User::create([
    'Nombres' => $request->nombres,
    'Apellidos' => $request->apellidos,
    'Carnet_Identidad' => $request->
        carnetIdentidad,
    'Contraseña' => Hash::make($request->
        contraseña),
    'Id_Rol' => $request->rolId,
    'Estado' => 'Activo',
]);

```

Fuente: Elaboración propia.

3.6.4.1 Eliminación simulada (Soft Delete)

Para mantener la trazabilidad histórica de las transacciones, el sistema no realiza un borrado físico de los registros de usuario. En su lugar, se implementa una eliminación lógica.^o *Soft Delete* manual, modificando la columna Estado a 'Inactivo'. Esto deshabilita al usuario del sistema sin perder sus registros relacionados:

Figura 3.16: Lógica de eliminación lógica

```

public function destroy($id)
{
    $user = User::findOrFail($id);
    $user->update(['Estado' => 'Inactivo'
    ]);
    return response()->json(['message' =>
        'Usuario desactivado'], 200);
}

```

Fuente: Elaboración propia.

3.7 Validación y manejo de errores

La robustez del sistema se garantiza mediante una estrategia de validación de doble capa (cliente y servidor) y un flujo estandarizado para la captura y presentación de errores, asegurando la integridad de los datos y una buena experiencia de usuario.

3.7.1 Validación de formularios

La validación de la información introducida por el usuario se divide en dos frentes:

- **Capa del Cliente (*Frontend*):** Se utiliza la integración de **React Hook Form** junto con **Zod**. React Hook Form gestiona el estado de las entradas minimizando los re-renderizados, mientras que Zod proporciona validación de esquemas con tipado estricto (*type safety*). Los esquemas, como `usuario.schema.ts`, definen reglas precisas (longitud mínima, formatos de correo, campos obligatorios) y bloquean el envío del formulario si no se cumplen, proporcionando retroalimentación visual inmediata.
- **Capa del Servidor (*Backend*):** Laravel actúa como la fuente de verdad definitiva. Utiliza sus clases `FormRequest` o validaciones integradas en los controladores para verificar nuevamente los datos entrantes. Esto previene la inyección de datos maliciosos en caso de que las validaciones del cliente sean eludidas.

3.7.2 Manejo de excepciones

El manejo de excepciones se ha estructurado para interceptar fallos tanto lógicos como de red, evitando que la aplicación se detenga abruptamente:

- **Excepciones de Red y API:** En el cliente, el servicio centralizado (`apiClient`) captura códigos de estado HTTP de error (4xx y 5xx). Si la respuesta indica un fallo (ej. `!res.ok`), se lanza una excepción controlada (`throw new Error(...)`) que interrumpe el flujo normal.
- **Excepciones de Tipado de Datos:** Se implementa un bloque `try-catch` al procesar las respuestas del servidor. Si la estructura del JSON devuelto por Laravel no coincide con el esquema esperado en Next.js, Zod lanza un `ZodError`, el cual se registra en la consola para facilitar la depuración, previniendo fallos silenciosos en la interfaz.

3.7.3 Control de errores

Una vez capturada una excepción, el sistema debe reaccionar de forma predecible y amigable para el usuario final:

- **Límites de Error (*Error Boundaries*):** En Next.js, se hace uso de los archivos `error.tsx` en rutas específicas. Estos actúan como límites de error de React, aislando el fallo a un componente o vista particular en lugar de colapsar toda la

aplicación.

- **Respuestas estandarizadas del Servidor:** Laravel está configurado para devolver siempre las excepciones de la API en formato JSON estandarizado, incluyendo un mensaje descriptivo y, en caso de errores de validación (HTTP 422), un desglose de los campos afectados.

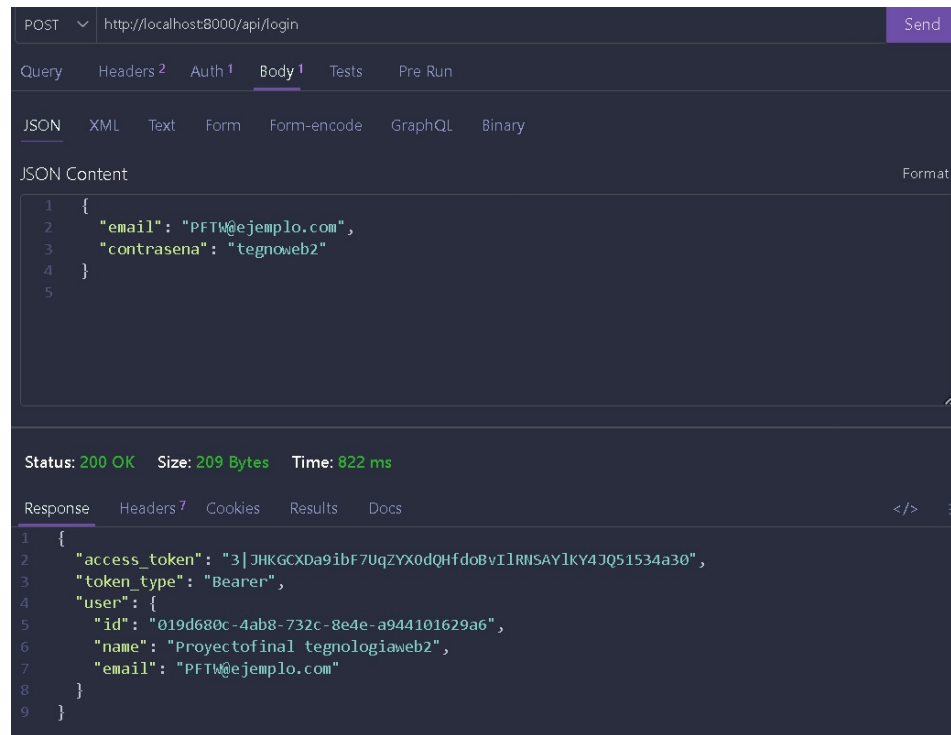
3.8 Pruebas del sistema

3.8.1 Pruebas de funcionamiento

Para validar la correcta operatividad y la integración entre el *frontend* y el *backend*, se ejecutaron pruebas funcionales enfocadas en los flujos críticos de negocio:

- **Pruebas de integración de la API:** Se utilizaron clientes HTTP (como Postman o el cliente integrado en Neovim) para atacar directamente los *endpoints* de Laravel. Se verificó el correcto funcionamiento de las operaciones CRUD, la emisión y revocación de *tokens* con Sanctum, y la coherencia de los datos almacenados en la base de datos PostgreSQL.

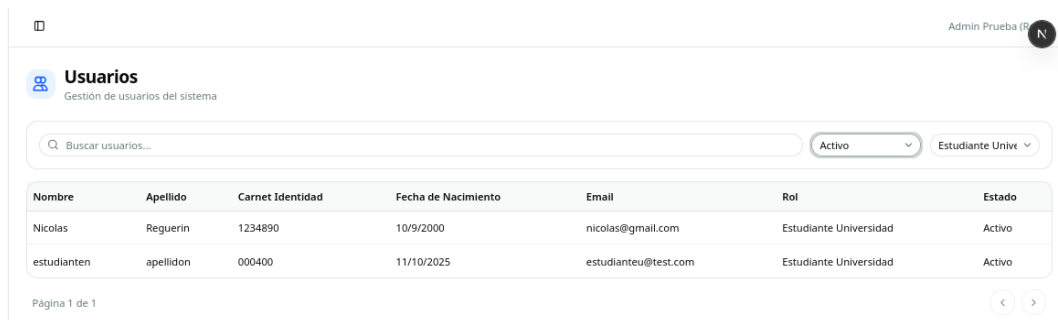
Figura 3.17: Verificación de respuesta exitosa (200 OK) y estructura JSON en el *endpoint* de usuarios.



Fuente: Elaboración propia.

- **Pruebas de interfaz (*End-to-End* manuales):** Se simuló el comportamiento de los roles administrativos interactuando con los *dashboards* de Next.js. Esto incluyó la verificación de persistencia de sesiones, el funcionamiento de los filtros y paginación en las tablas de datos (ej. tabla de transacciones), y la correcta visualización de mensajes de error al forzar escenarios anómalos.

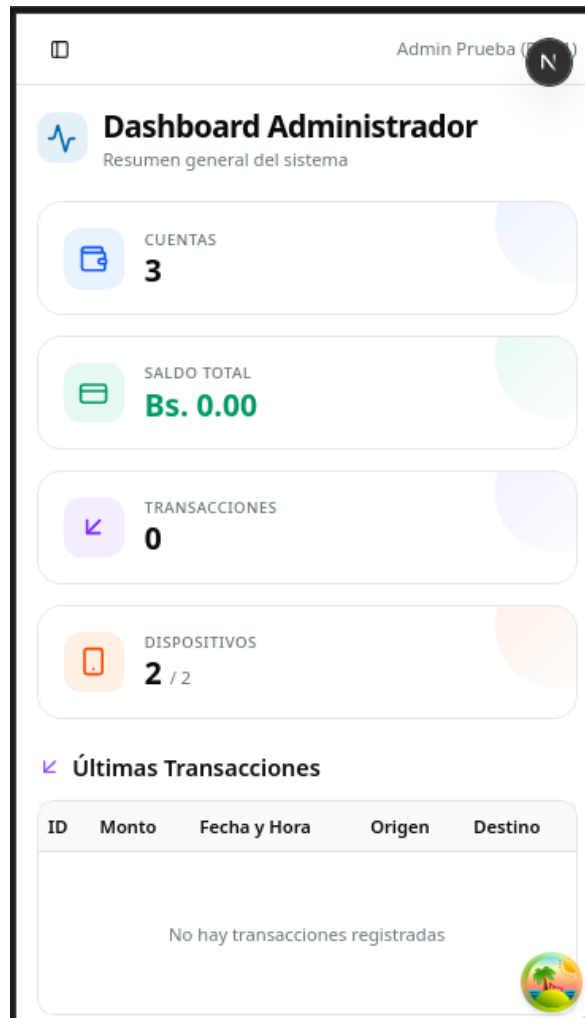
Figura 3.18: Interfaz del *dashboard* de usuarios mostrando el funcionamiento de filtros dinámicos y paginación.



Fuente: Elaboración propia.

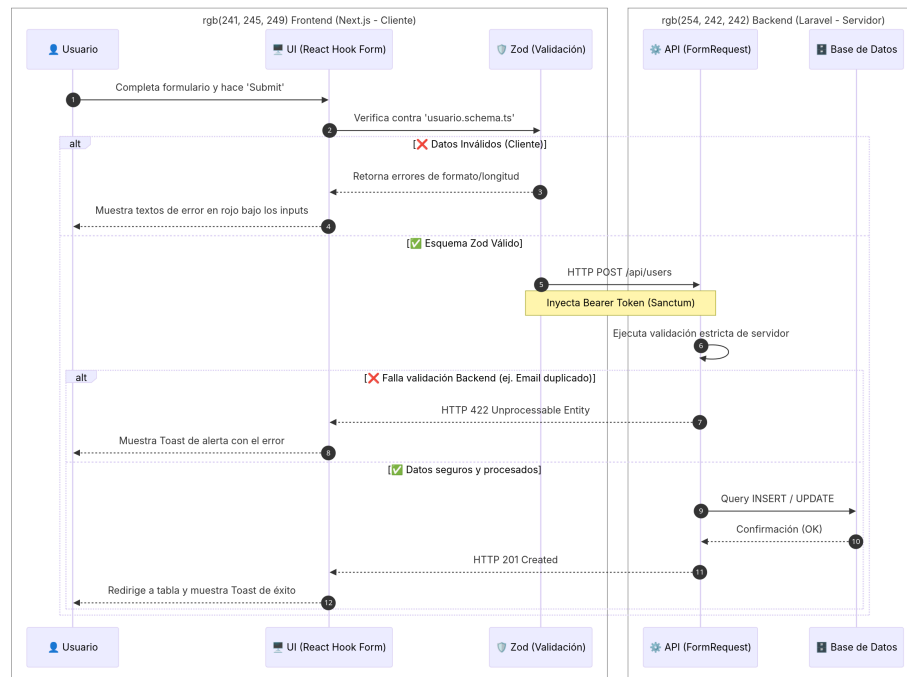
- **Verificación de accesibilidad y adaptabilidad:** Se constató que los componentes visuales respondan correctamente a diferentes resoluciones (diseño *responsive*), validando el uso del *hook* personalizado `use-mobile.ts` en dispositivos móviles.

Figura 3.19: Visualización de la interfaz en entorno móvil con el menú lateral colapsado automáticamente.



Fuente: Elaboración propia.

Figura 3.20: Diagrama de flujo de validación de datos entre cliente y servidor.



Fuente: Elaboración propia.

3.9 Resultados

3.9.1 Funcionamiento final

El desarrollo del sistema concluyó con la exitosa integración de la arquitectura cliente-servidor propuesta. Se logró establecer una comunicación fluida y segura entre la interfaz de usuario (*frontend* en Next.js) y la API RESTful central (*backend* en Laravel).

Entre los resultados técnicos y funcionales más destacados de la implementación se encuentran:

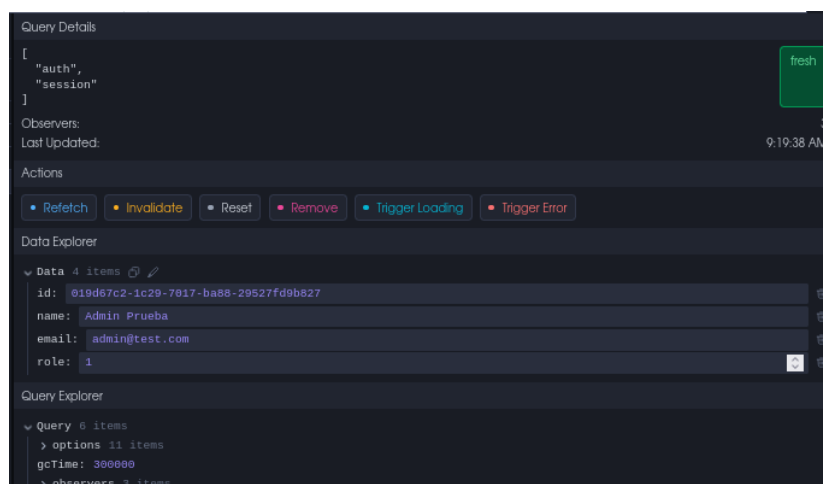
- **Gestión integral de dominios:** Los módulos principales (usuarios, roles, dispositivos, cuentas y transacciones) operan correctamente de extremo a extremo. Las operaciones de lectura, escritura y actualización mantienen una estricta integridad referencial en la base de datos PostgreSQL.

- **Seguridad y control de sesiones:** El flujo de autenticación basado en *tokens* (vía Laravel Sanctum) protege eficazmente las rutas del cliente. El enrutamiento de Next.js bloquea exitosamente el acceso a los *dashboards* administrativos si no existe una sesión activa y válida.
- **Rendimiento y experiencia de usuario (UX):** La arquitectura de Next.js, combinando *Server Components* para la estructura base y *Client Components* para interactividad (como las tablas de datos de transacciones), resultó en tiempos de carga óptimos. Además, el sistema de validación de doble capa con Zod demostró ser efectivo para prevenir la entrada de datos corruptos sin degradar la experiencia del usuario.

3.9.2 Capturas del sistema

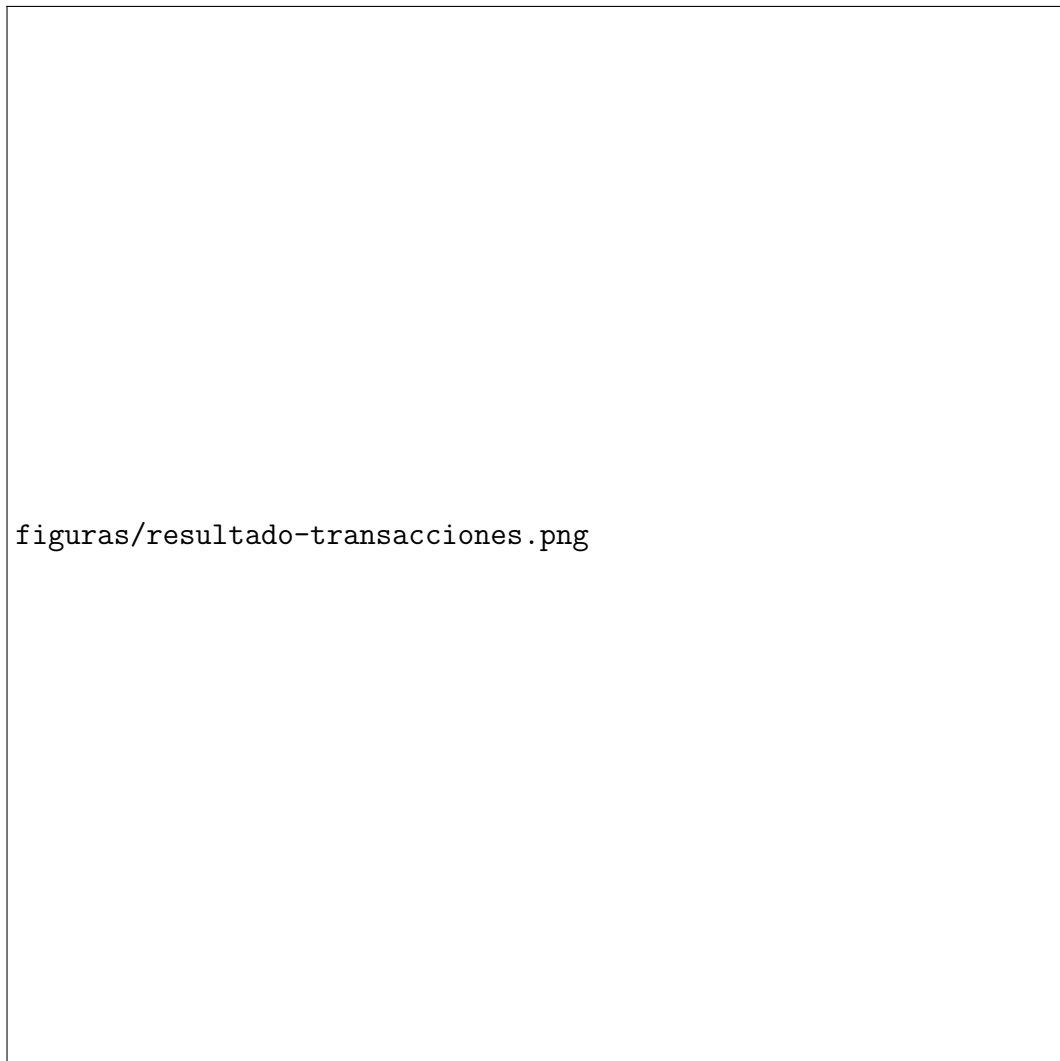
A continuación, se presenta evidencia visual de los componentes e interfaces más representativos del sistema en funcionamiento. Las siguientes imágenes demuestran la aplicación consistente del sistema de diseño (Tailwind CSS y *shadcn/ui*) y la disposición de las herramientas de gestión.

Figura 3.21: Interfaz de autenticación del sistema, con validación de credenciales en tiempo real.



Fuente: Elaboración propia.

Figura 3.22: Módulo de transacciones (*Data Table*) mostrando el listado de operaciones, con soporte para búsqueda dinámica y paginación.



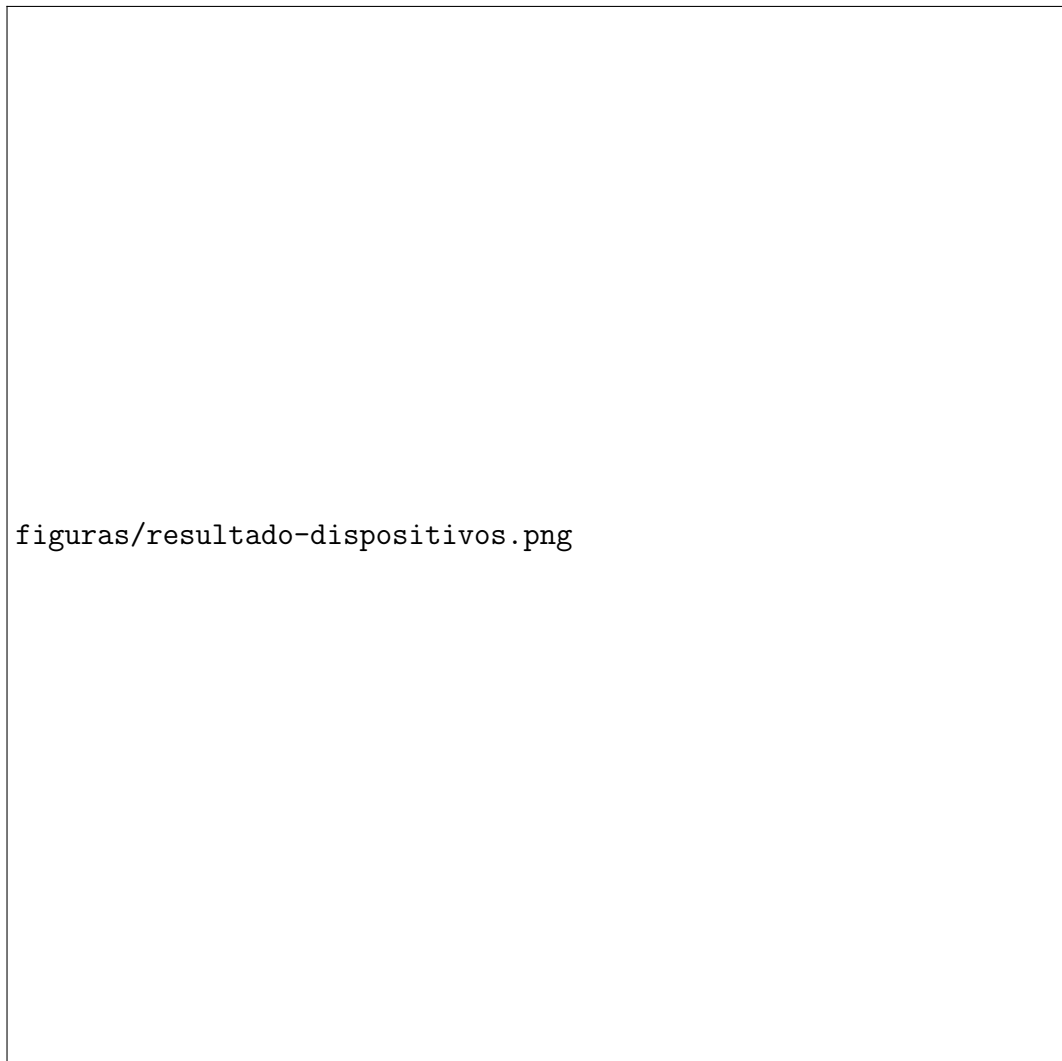
Fuente: Elaboración propia.

Figura 3.23: Panel interactivo para la gestión de roles, demostrando el uso de modales laterales (*Sheets*) para la inserción de datos sin abandonar el contexto de la tabla.



Fuente: Elaboración propia.

Figura 3.24: Vista de gestión de dispositivos móviles y tarjetas NFC vinculadas a las cuentas de los usuarios.



Fuente: Elaboración propia.

4. Conclusiones y recomendaciones

Redacción de conclusiones...

Referencias

Docker Inc. (2023). Docker documentation.

ECMA International (2017). The json data interchange standard.

ECMA International (2023). EcmaScript language specification.

Erl, T. (2005). *Service-Oriented Architecture*. Prentice Hall.

Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures.

Gamma, E., Helm, R., Johnson, R., and Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.

Garrett, J. (2005). Ajax: A new approach to web applications.

Kurose, J. and Ross, K. (2017). *Computer Networking: A Top Down Approach*. Pearson.

Laravel (2023). Laravel documentation.

Meta Platforms, Inc. (2023). React documentation.

Microsoft Corporation (2023). Typescript documentation.

Oracle (2023). Mysql documentation.

PHP Group (2023). Php manual.

Pressman, R. (2014). *Ingeniería de Software*. McGraw-Hill.

Tailwind Labs Inc. (2023). Tailwind css documentation.

Tanenbaum, A. (2016). *Computer Networks*. Pearson.

Vercel, Inc. (2023). Next.js documentation.